

Project Management AI Assistant System Design and Prototype Architecture

Talib

December 18, 2025

Contents

1	Executive summary	3
2	System scope	3
2.1	Target capabilities	3
2.2	Out of scope in the current prototype	3
3	Data model	3
4	Architecture overview	4
4.1	Components	4
4.2	High-level architecture diagram	5
5	Agent orchestration with LangGraph	5
5.1	State model	5
5.2	Graph structure	6
6	Request flows	6
6.1	Read flow	6
6.2	Write flow with approval and role enforcement	7
7	Production-grade concerns	7
7.1	Authentication and session integrity	7
7.2	Rate limiting and abuse controls	7
7.3	Error handling and graceful degradation	7
7.4	Write safety and schema validation	8
8	Scalability and extensibility	8
8.1	Scaling to more users and concurrent chats	8
8.2	Adding integrations and new tools	8
9	Evaluation strategy	8
9.1	Current evals (implemented)	8
9.2	Recommended eval expansion (next iteration)	8
9.3	Concrete eval cases	9
10	Trade-offs and limitations	9
10.1	Why LangGraph	9
10.2	Limitations in the current implementation	9

10.3 Forward-looking improvements	10
11 Loom video script (5 to 8 minutes)	10

1 Executive summary

This system is a chat-based Project Management AI Assistant that reads and, with explicit admin approval, writes to a relational project management database. The prototype uses a Next.js frontend for login and chat, and a FastAPI backend hosting a LangGraph agent. The agent classifies intent, calls structured read and write tools backed by Async SQLAlchemy, and uses a human-in-the-loop interrupt to guarantee that write operations never execute without an explicit approval action.

The architecture is designed to be defensible in production discussions: JWT authentication, role-based write restrictions (admin-only), rate limiting via SlowAPI, request ID propagation, structured logging, and graceful degradation when optional components are not available. LangGraph checkpointing enables resumable multi-step workflows and allows the approval gate to pause execution safely.

The evaluation approach is intentionally lightweight but extensible: a YAML-driven intent routing harness is included, and the design supports expanding evals to tool-call correctness, approval-gating correctness, workflow completion, and regression detection in CI.

2 System scope

2.1 Target capabilities

The implemented scope aligns to the assessment requirements:

- **Read:** query tasks, projects, and team members using structured tools (for example, “List tasks assigned to Sarah”).
- **Write with explicit approval:** create and update projects and tasks, but only after an admin approves the proposed plan.
- **Workflow:** multi-step requests are handled as write proposals that are paused prior to execution, enabling a single approval for a compound plan.
- **Context-aware:** the router supports CLARIFY and the system prompt instructs the assistant to ask clarifying questions when entity references are ambiguous.
- **Evals:** a runnable eval script validates intent routing against a small dataset, and the repository includes additional smoke tests.

2.2 Out of scope in the current prototype

This prototype does not implement fine-grained permissions per entity or field. Instead, it uses a strict policy: any write requires an admin role. This is deliberate because it prevents accidental mutation during evaluation. In a production iteration, role-based access control would be defined as an explicit policy table and enforced consistently across prompt, tools, and API.

3 Data model

The backend uses SQLAlchemy ORM models for `Project`, `Task`, and `TeamMember`. Projects and tasks reference team members for ownership and assignment.

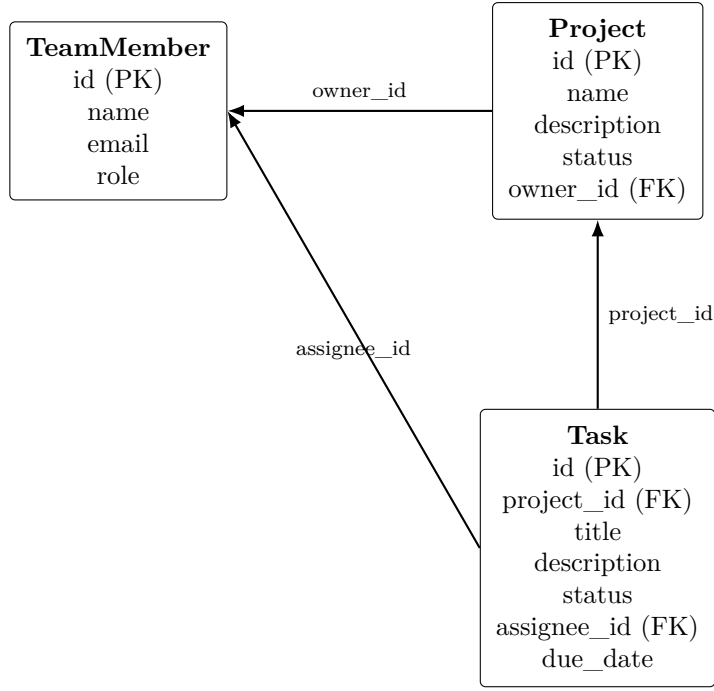


Figure 1: Entity relationship view of the core data model.

4 Architecture overview

4.1 Components

- **Next.js frontend:** login page, chat page, approval prompt UI, and session storage for JWT.
- **FastAPI backend:** authentication endpoints, chat endpoints, global exception handling, CORS, and rate limiting.
- **LangGraph runtime:** orchestrates intent routing, read tool execution, write planning, approval pause, and write execution.
- **Tool layer:** typed tools for reads (`get_tasks`, `get_projects`, `get_team_members`) and writes (`create_project`, `create_task`, `update_project`, `update_task`).
- **Primary datastore:** relational database accessed via async SQLAlchemy sessions.
- **Checkpoint store:** LangGraph checkpointer in SQLite for durable conversation state per thread.
- **Observability:** request ID propagation, structured logs, latency capture, and rate-limit metrics via middleware.
- **Evals harness:** YAML-driven intent tests executed via `python -m app.evals.runner`.

4.2 High-level architecture diagram

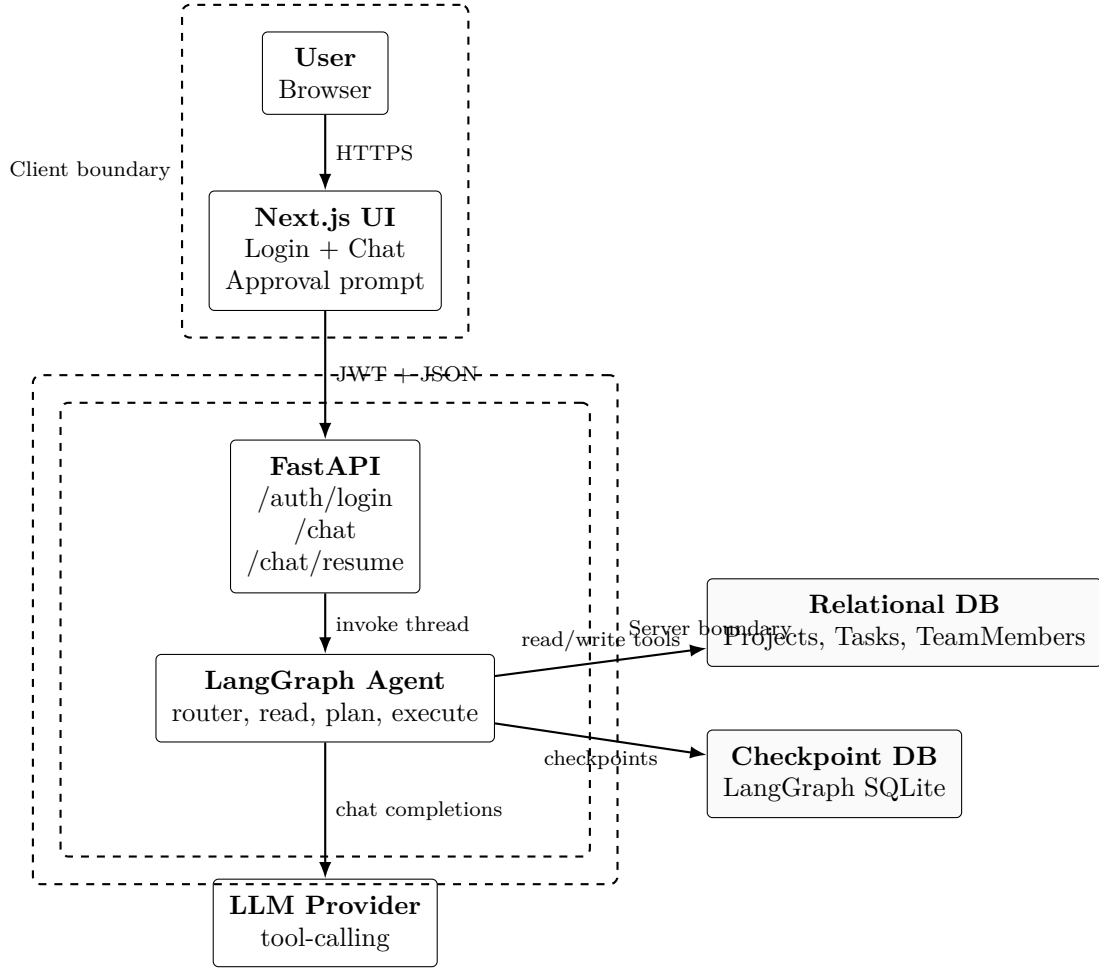


Figure 2: End-to-end vertical architecture with explicit trust boundaries and state persistence.

5 Agent orchestration with LangGraph

5.1 State model

The LangGraph state is a dictionary (**AgentState**) that includes:

- **messages**: the conversation history as a list of user and assistant messages.
- **user_info**: injected by the API layer and used by the session guard.
- **intent**: the router output (READ_QUERY, WRITE_PROPOSAL, WORKFLOW, CLARIFY).
- **approved**: tri-state approval flag (None, True, False).
- **approval_token**: server-generated token to bind the resume call to the paused graph state.

This state design keeps the backend stateless with respect to the chat, because conversation state lives in LangGraph checkpoints keyed by **thread_id**.

5.2 Graph structure

The graph uses explicit nodes for guarding sessions, routing intent, executing reads with tool loops, planning writes, and executing writes. The graph is compiled with `interrupt_before=["write_executor"]`, which pauses execution prior to any mutation.

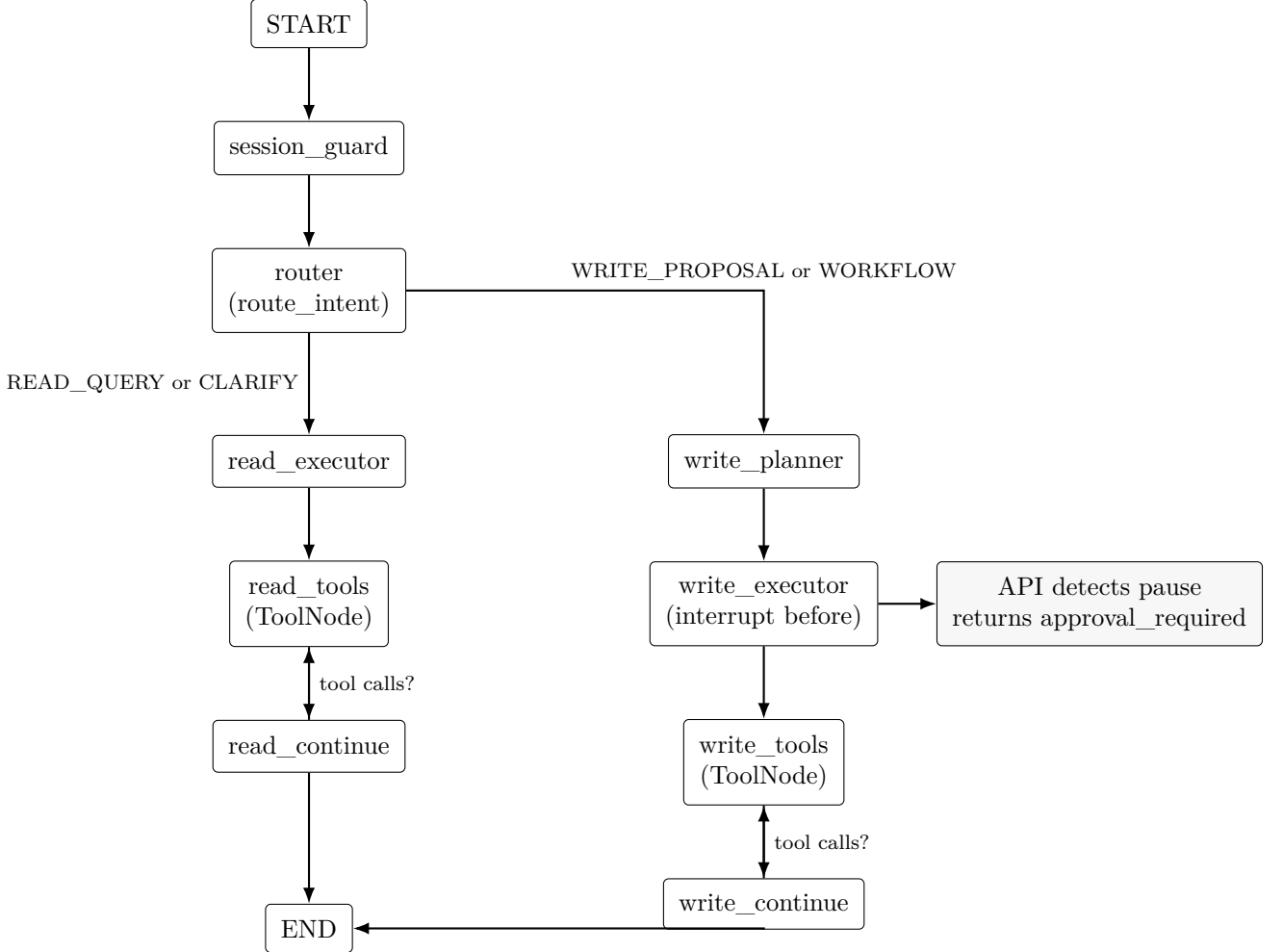


Figure 3: Vertical LangGraph control flow with explicit read, write, and approval interrupt paths.

6 Request flows

6.1 Read flow

A read request is executed end-to-end in a single `/chat` call.

1. API validates JWT, fetches the user, and injects `user_info`.
2. Router classifies intent as `READ_QUERY`.
3. `read_executor` calls the tool-enabled model. If the model requests tools, the `ToolNode` executes them.
4. `read_continue` allows the model to synthesise an answer from tool outputs.
5. The backend returns a `type=message` payload.

Key safety choices in the read tools:

- Each tool caps `limit` (tasks and projects capped at 100, members capped at 50) to avoid runaway responses.
- LIKE queries sanitise wildcard characters to reduce accidental broad scans.
- Errors are caught and returned as structured error strings or objects, preventing unhandled exceptions from leaking internal traces.

6.2 Write flow with approval and role enforcement

Write requests follow a two-step pattern:

1. **Plan:** `/chat` routes to `write_planner` which produces a proposed change plan and asks for approval.
2. **Pause:** the graph is interrupted before `write_executor`. The API detects `snapshot.next` contains `write_executor`.
3. **Authorise:** if the user is not an admin, the API rejects the request without resuming the graph.
4. **Bind approval:** for admins, the API stores an `approval_token` in graph state and returns `type=approval_required`.
5. **Resume:** `/chat/resume` checks token match and sets `approved=True` or `approved=False`.
6. **Execute:** on approval, `write_executor` runs tool calls (create or update). On rejection, no tools are executed.

This design is robust because approval is enforced at two layers: LangGraph interrupt stops execution, and the API enforces role checks and token binding before resuming.



Figure 4: Write approval sequence at a high level (plan, pause, token, resume, execute).

7 Production-grade concerns

7.1 Authentication and session integrity

The backend issues JWTs containing user id, email, and role. Every chat call requires a bearer token, and the backend re-loads the user from the database and validates that email and role match the token payload. This prevents stale tokens from being used after a role change.

7.2 Rate limiting and abuse controls

The backend uses SlowAPI rate limits for login and chat endpoints. This protects the LLM provider from accidental or malicious request floods and provides a clear control point for operational tuning.

7.3 Error handling and graceful degradation

The API uses a global exception handler that returns a stable JSON error format and includes a request id for correlation. Middleware logs request latency, status codes, and database pool status.

The application lifespan initialises both the database and the LangGraph checkpointer. If the persistent checkpointer fails, it falls back to an in-memory checkpointer. If the database fails and `DB_REQUIRED=false`, the service continues to run and returns a readiness payload that indicates database disconnection.

7.4 Write safety and schema validation

Write tools restrict patch fields with explicit allow-lists to prevent arbitrary field updates. Input validation is performed in Pydantic request models at the API edge and again within tools by checking entity existence and allowed fields.

8 Scalability and extensibility

8.1 Scaling to more users and concurrent chats

The backend remains horizontally scalable because it does not keep chat state in process memory. Conversation state is stored in the LangGraph checkpoint store keyed by `thread_id`. In production, the checkpoint store should move from SQLite to a networked store (for example Postgres) to support multiple replicas safely.

Concurrency concerns:

- Tool execution uses async SQLAlchemy sessions per request.
- Rate limiting provides back-pressure.
- The design supports adding a task queue for heavy workflows without changing the agent interface.

8.2 Adding integrations and new tools

New integrations should be added as additional tools with strict input schemas. The router can be extended to add new intents or to route by tool selection. Because tools are bound separately for read and write models, capability expansion does not require rewiring the whole graph.

A practical next step is adding a dedicated “project lookup” tool and a workflow node that resolves entity ids before proposing a plan. This reduces the risk of plans that reference non-existent projects.

9 Evaluation strategy

9.1 Current evals (implemented)

The repository includes a YAML-based routing evaluation runner that checks that `route_intent` produces the expected intent for a small set of inputs.

9.2 Recommended eval expansion (next iteration)

To make regressions obvious, the eval suite should cover:

- **Tool-call correctness:** given a prompt, assert that the model selects the correct tool and produces valid arguments.
- **Approval gating correctness:** ensure write tools are never called without `approved=True`.
- **Workflow completion:** verify multi-step creation returns created ids and the right number of tasks.

- **RBAC enforcement:** verify non-admins always receive a denial response on write attempts.
- **Response quality:** enforce stable formatting, presence of ids, and clear next steps.

9.3 Concrete eval cases

Table 1 lists a minimal set of test cases that can be implemented with tool mocking and deterministic seeds.

Table 1: Example eval cases for regression detection.

User input	Expected	Assertions
List my tasks	READ_QUERY	Calls <code>get_tasks</code> with <code>assignee_email</code> inferred or asks clarify.
Show all projects	READ_QUERY	Calls <code>get_projects</code> with <code>limit</code> capped.
Who is Bob?	READ_QUERY	Calls <code>get_team_members(name_or_email="Bob")</code> and summarises.
Create project Beta owned by bob@example.com	WRITE_PROPOSAL	Returns <code>approval_required</code> for admin; no DB mutation before resume.
Create project Beta as non-admin	WRITE_PROPOSAL	Returns access denied message; graph not resumed.
Mark task 12 as done	WRITE_PROPOSAL	On approval: calls <code>update_task(task_id=12, patch={status})</code> .
Add task “QA” to project 9999	WORKFLOW or WRITE_PROPOSAL	Tool returns “project not found”; assistant surfaces error and asks next step.
Assign task 5 to unknown user	WRITE_PROPOSAL	Tool warns assignee not found; assistant requests valid email.
Change owner of project 1 to jane@example.com	WRITE_PROPOSAL	Requires approval; patch allow-list enforced.
What is the revenue of X?	READ_QUERY	Calls <code>get_financial_data</code> instead of project tools.

10 Trade-offs and limitations

10.1 Why LangGraph

LangGraph is used because it provides explicit state, persistence, interrupts, and re-entrancy. A simple loop would be faster to write but would be harder to pause safely and resume deterministically after approval.

10.2 Limitations in the current implementation

- The write planner relies on free-form text rather than a strictly validated JSON schema. A stronger approach is to force a JSON schema output and validate it server-side.
- Role checks are strict (admin-only) and should be expanded into a clear policy matrix.
- The checkpoint store uses SQLite, which is not suitable for multi-replica deployments without careful coordination.

10.3 Forward-looking improvements

The most valuable next changes are:

- Introduce a structured “ProposedChange” schema and validate before offering approval.
- Add an explicit audit log table for writes (who approved, what changed, before and after, request id).
- Move checkpoint persistence to a shared store and enable horizontal scaling.
- Expand evals to include tool-call traces and approval gating assertions in CI.

11 Loom video script (5 to 8 minutes)

Minute 0 to 1: framing

Show the repository structure. Explain the goal: chat assistant for project management, safe writes with approval, and reliable orchestration.

Minute 1 to 3: architecture and graph

Show the backend `app/main.py` lifespan initialisation, then `app/agent/graph.py`. Explain the interrupt-before-write design and how the API detects paused execution via `aget_state`.

Minute 3 to 6: demo

Run three demo prompts:

- Read: “What tasks are assigned to Sarah?”
- Write: “Mark task 12 as done” (show approval, then approve)
- Workflow: “Create a project called Client Portal owned by Sarah and add Design, Build, QA” (show single approval then execution)

Minute 6 to 8: evals and scaling

Show `app/evals/runner.py` and `cases.yaml`. Explain how you would extend evals to include tool call traces and approval assertions. Close with scaling plan: shared checkpoint store, stateless replicas, and observability improvements.